- 11 参数验证与版本治理
  - 11.1 运行场景
  - 11.2 决策困难与算法需求
  - 11.3 验证指标
  - 11.4 参数发布卡
  - 11.5 手算示例
  - 11.6 核心实现
  - 11.7 适用域与回滚
  - 11.8 业务决策
  - 11.9 自测

11 参数验证与版本治理

一句话：一个浮点数不是参数；可用参数必须带适用工况、验证残差、版本和回滚条件，否则无法支撑回放与长期维护。

11.1 运行场景

17:00，EMS 准备使用下午刚辨识出的 `R0`、`R_d`、`τ` 和 `C_eff` 计算晚峰能力。算法输出的数值看起来合理，但值班员还必须确认：这些参数是否来自与当前温度、SOC 区间和负载倍率相近的数据？它们在未参与拟合的时间段中是否仍能解释端电压？

如果直接发布一组只在训练数据上表现良好的参数，17:30 的 EKF 可能将模型偏差吸收到 SOC；若缺少参数版本，21:00 回放时也无法判断当时使用了哪套能力依据。

11.2 决策困难与算法需求

参数不是孤立浮点数，而是带有适用域、数据来源、残差证据、生效时间和回滚条件的运行资产。EMS 需要判断"当前工况能否使用这组参数"，而不是只判断"这组参数是否算出来了"。

因此需要训练/验证分离、残差边界、温度与 SOC 适用域、版本发布和回滚规则。该算法让第 17–22 章使用的模型具有明确来源，使运行日中的异常可以追溯到具体参数版本。

11.3 验证指标

```
RMSE = √( mean_i (y_i − ŷ_i)² )     （拟合/泛化误差，mV）
gap  = val_RMSE − train_RMSE        （泛化差距，mV）
bias = mean_i (y_i − ŷ_i)           （残差均值，检查系统性偏差，mV）
```

判据：验证 RMSE 不应显著高于训练 RMSE（`gap` 接近或小于零）；`|bias|` 应接近零；参数必须落在物理边界内。

第 08 章的结果为：

```
train RMSE=0.644 mV
validation RMSE=0.603 mV
validation mean=-0.060 mV
```

验证误差没有高于训练误差，残差均值接近零，可进入发布审查。

11.4 参数发布卡

字段	示例	作用
参数集	$R_0=0.3216\ m\Omega$, $R_d=0.1988\ m\Omega$, $\tau=20.5\ s$	模型本体
有效容量	$C_{eff}=313.9\ Ah$, $SOH=99.97\%$	SOC 与能量换算分母
适用工况	20–35°C，15–85% SOC，0.1–0.5C	适用域
训练/验证集	PulseSet-2026Q2-A/B	数据来源
验证残差	val RMSE 0.603 mV，mean −0.060 mV	质量证据
版本/生效	ECM-LFP-314Ah-v1.3，2026-07-01	可追溯
回滚条件	NIS 连续超限、温度超域、参数漂移超阈	运行退路

11.5 手算示例

某候选参数集的训练/验证残差如下。

第 1 步：泛化差距。

```
gap = val_RMSE − train_RMSE = 0.603 − 0.644 = −0.041 mV
```

第 2 步：偏差检查。

```
|bias| = |val_mean| = 0.060 mV < 0.5 mV
```

两项通过，可发布 v1.3；否则保留上一版已验证参数并告警。

11.6 核心实现

下面函数把验证结果、参数边界和工况范围汇总为发布结论。它是可直接运行的教学版发布门控。

```python
from dataclasses import dataclass


@dataclass(frozen=True)
```

```python
class ReleaseDecision:
    approved: bool
    reasons: tuple[str, ...]


def assess_release(train_rmse_mv: float, validation_rmse_mv: float,
                   validation_mean_mv: float, r0_mohm: float,
                   temperature_c: float, soc: float) -> ReleaseDecision:
    """检查泛化差距、残差偏差、R0 边界与适用工况。"""
    reasons = []
    if train_rmse_mv <= 0 or validation_rmse_mv <= 0:
        reasons.append("RMSE 必须为正")
    elif validation_rmse_mv > train_rmse_mv * 1.5:
        reasons.append("验证误差显著高于训练误差")
    if abs(validation_mean_mv) > 0.5:
        reasons.append("残差均值超出 0.5 mV")
    if not 0.1 <= r0_mohm <= 0.8:
        reasons.append("R0 超出 0.1–0.8 mΩ 边界")
    if not 20.0 <= temperature_c <= 35.0:
        reasons.append("温度超出 20–35°C 适用域")
    if not 0.15 <= soc <= 0.85:
        reasons.append("SOC 超出 15–85% 适用域")
    return ReleaseDecision(not reasons, tuple(reasons))


if __name__ == "__main__":
    decision = assess_release(0.644, 0.603, -0.060, 0.3216, 25.0, 0.50)
    print(decision)
    assert decision.approved

    rejected = assess_release(0.5, 8.0, 0.1, 0.3216, 25.0, 0.50)
    assert not rejected.approved
    assert "验证误差显著高于训练误差" in rejected.reasons
```

11.7 适用域与回滚

参数会随 SOC、温度和 SOH 变化。超出适用域时，应重新辨识或采用保守参数；不能把一个局部工况的参数当作全场景常数。

递推最小二乘（RLS）可在线更新线性参数，其中 $e[k]=y[k]-\phi[k]^{\mathsf{T}}\theta[k-1]$ 为第 k 拍参数残差：

$$
\begin{aligned}
e[k] &= y[k] - \phi[k]^{\mathsf{T}}\theta[k-1] && \text{（参数残差，V）}\\
K[k] &= P[k-1]\phi[k] \,/\, (\lambda + \phi[k]^{\mathsf{T}}P[k-1]\phi[k])\\
\theta[k] &= \theta[k-1] + K[k]\cdot e[k]\\
P[k] &= (P[k-1] - K[k]\phi[k]^{\mathsf{T}}P[k-1]) \,/\, \lambda
\end{aligned}
$$

在线更新应同时满足质量门控、持续激励、参数投影、变化率限制和影子验证；没有足够激励时冻结更新。

11.8 业务决策

17:00 参数发布通过后，EMS 将参数版本连同适用温度、SOC 区间和残差指标传给状态与能力计算。
17:30 若温度超域或 NIS 持续异常，系统回滚至上一版已验证参数或进入保守能力模式；这样第 25 章能追溯每次任务使用的模型依据。

11.9 自测

1. 训练 RMSE 远小于验证 RMSE 说明什么？该发布吗？
2. 为什么发布参数必须带适用工况和版本？
3. 超出适用温度域时，正确动作是什么？

▶ 参考答案

1. 说明可能过拟合或工况漂移；不应发布，应调查数据与分层。
2. 否则无法在回放中复现，也无法判断当前工况是否适用或回滚。
3. 重新辨识该工况参数，或按保守策略降级并记录。